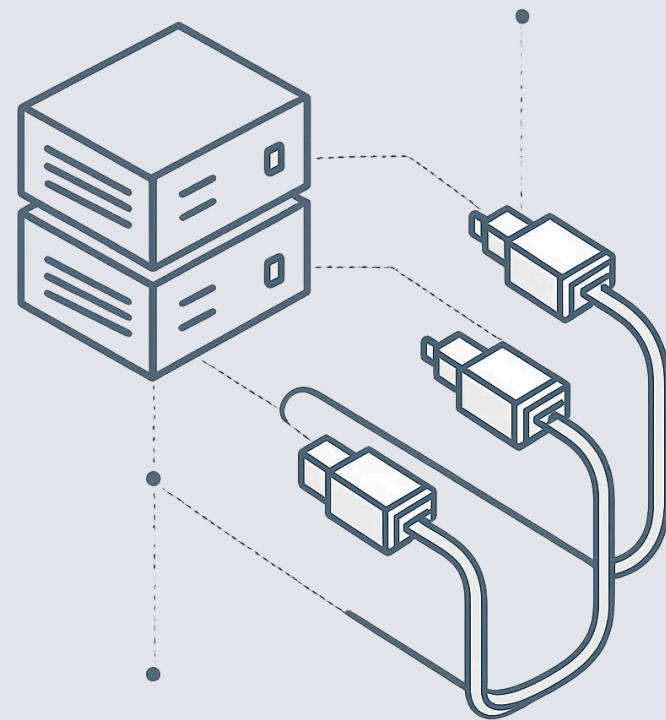


Spring WebSockets – Module Avancé



Au programme

Ce que nous allons couvrir

01

Architecture temps réel

Patterns et bonnes pratiques pour concevoir des systèmes réactifs robustes

03

Broadcasting ciblé

Diffusion globale, par topic, par utilisateur et par groupe

02

Sessions & Connexions

Gestion du cycle de vie, déconnexions et stratégies de reconnexion

04

UI réactive & Monitoring

Interfaces dynamiques et supervision des connexions en production

Rappel

Acquis du module précédent

WebSocket

Protocole full-duplex
sur TCP, connexion
persistante

STOMP

Protocole de
messagerie par-
dessus WebSocket

SockJS

Fallback HTTP pour
les environnements
restrictifs

Broker

Routeur de messages in-memory ou externe (RabbitMQ)

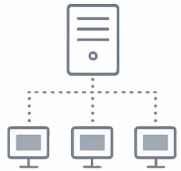




Architecture des Applications Temps Réel

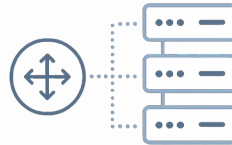
Concevoir un système capable de gérer des milliers de connexions simultanées nécessite une réflexion architecturale rigoureuse.

Patterns d'architecture temps réel



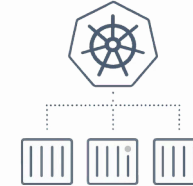
Monolithique

Un seul serveur Spring Boot gère tout.
Simple à déployer, limité en scalabilité.



Load Balanced

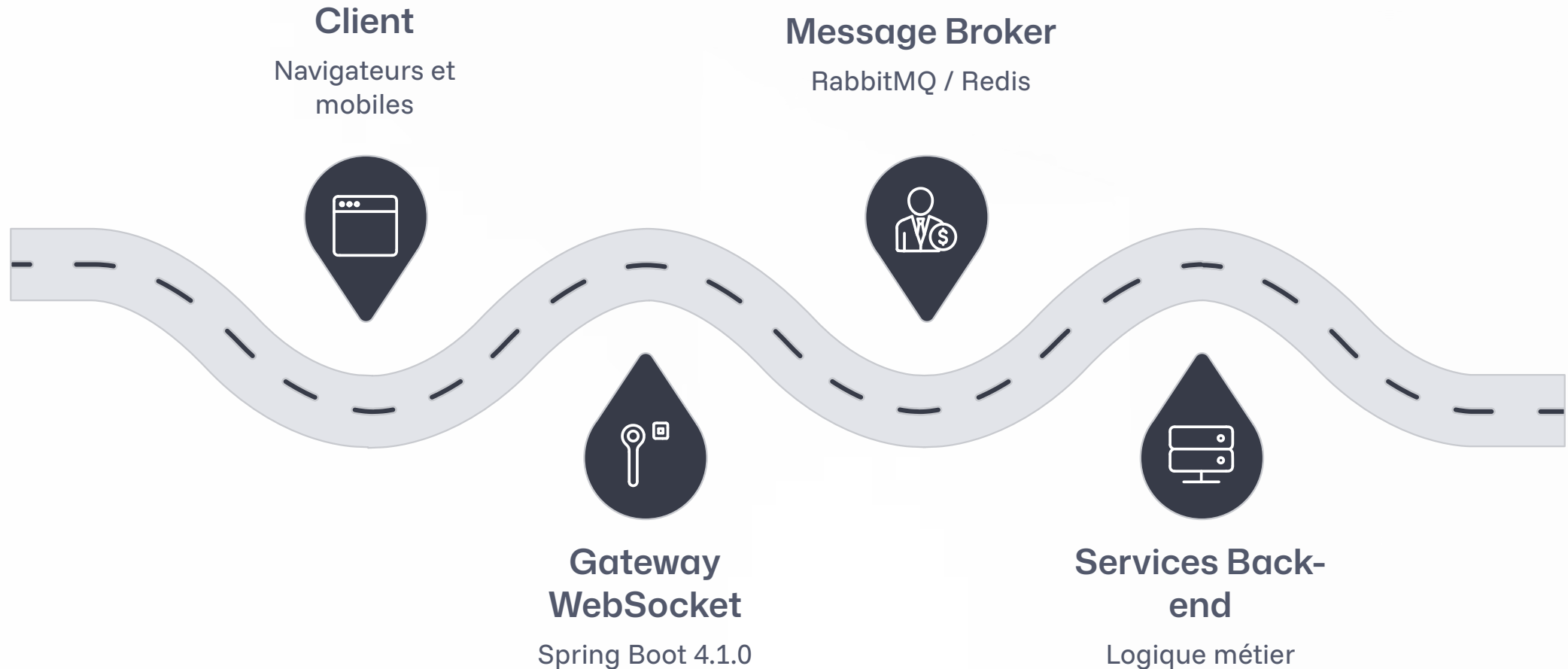
Plusieurs instances derrière un proxy.
Nécessite un **broker externe partagé**
(RabbitMQ, Redis).



Microservices

Chaque service publie/consomme des
événements via un bus de messages
centralisé.

Architecture recommandée avec Spring Boot 4.1.0



Le **Message Broker externe** est la clé : il permet la synchronisation entre plusieurs instances Spring Boot et garantit la livraison des messages même lors de redémarrages.

Configuration du broker externe

Avec Spring Boot 4.1.0, activer `StompBrokerRelay` pour déléguer au broker externe :

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements
    WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(
        MessageBrokerRegistry registry) {

        registry.enableStompBrokerRelay("/topic", "/queue")
            .setRelayHost("localhost")
            .setRelayPort(61613)
            .setClientLogin("guest")
            .setClientPasscode("guest");

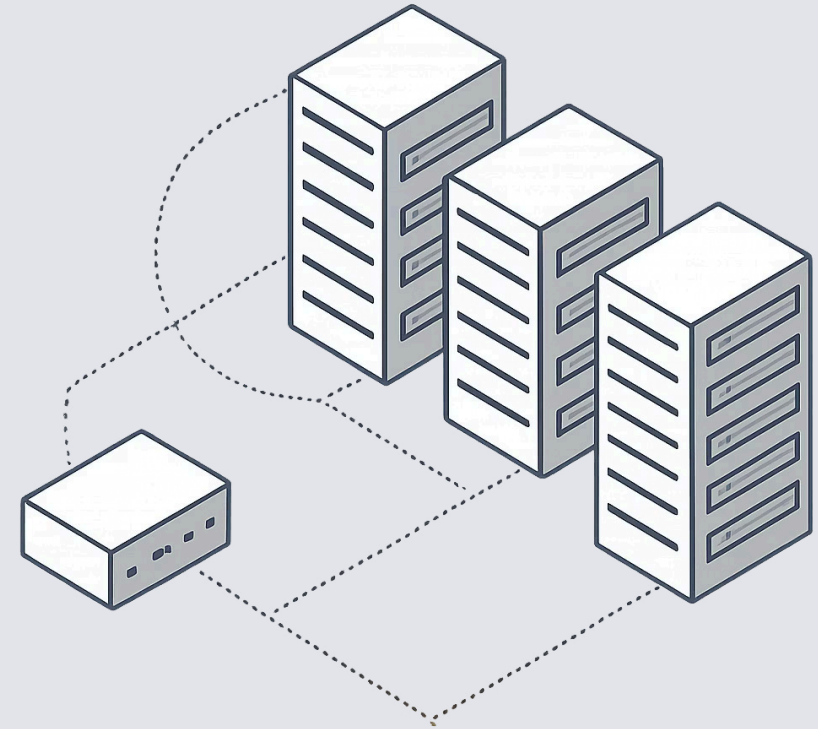
        registry.setApplicationDestinationPrefixes("/app");
    }
}
```

 Le relay STOMP délègue la gestion des abonnements et du routage à RabbitMQ. Spring Boot ne fait que passerelle.

 En multi-instances, le broker in-memory **ne suffit pas** : chaque instance n'a pas connaissance des sessions des autres.

Gestion des Sessions WebSocket

Chaque connexion WebSocket ouvre une session avec un identifiant unique. Comprendre ce cycle de vie est essentiel pour construire des applications fiables.



Cycle de vie d'une session WebSocket



Spring expose chaque étape via des **événements** `ApplicationContext` interceptables avec `@EventListener`.

Intercepter les événements de session

```
@Component
public class WebSocketSessionListener {

    private final Map<String, WebSocketSessionInfo> sessions
        = new ConcurrentHashMap<>();

    @EventListener
    public void handleSessionConnect(
        SessionConnectEvent event) {
        String sessionId = getSessionId(event);
        String user = getPrincipal(event);
        sessions.put(sessionId,
            new WebSocketSessionInfo(sessionId, user,
                Instant.now()));
        log.info("Connexion: {} ({})", user, sessionId);
    }

    @EventListener
    public void handleSessionDisconnect(
        SessionDisconnectEvent event) {
        sessions.remove(event.getSessionId());
        log.info("Déconnexion: {}", event.getSessionId());
    }
}
```

- ✔ Utilisez `ConcurrentHashMap` pour garantir la thread-safety lors d'accès simultanés depuis plusieurs threads WebSocket.

Stocker et interroger les sessions actives

```
@Service
public class SessionRegistry {

    private final Map<String, UserSession> activeSessions
        = new ConcurrentHashMap<>();

    public void register(String sessionId,
        String username) {
        activeSessions.put(sessionId,
            new UserSession(sessionId, username,
                Instant.now()));
    }

    public Set<String> getActiveUsers() {
        return activeSessions.values().stream()
            .map(UserSession::username)
            .collect(Collectors.toSet());
    }

    public long countActiveSessions() {
        return activeSessions.size();
    }
}
```

Cas d'usage

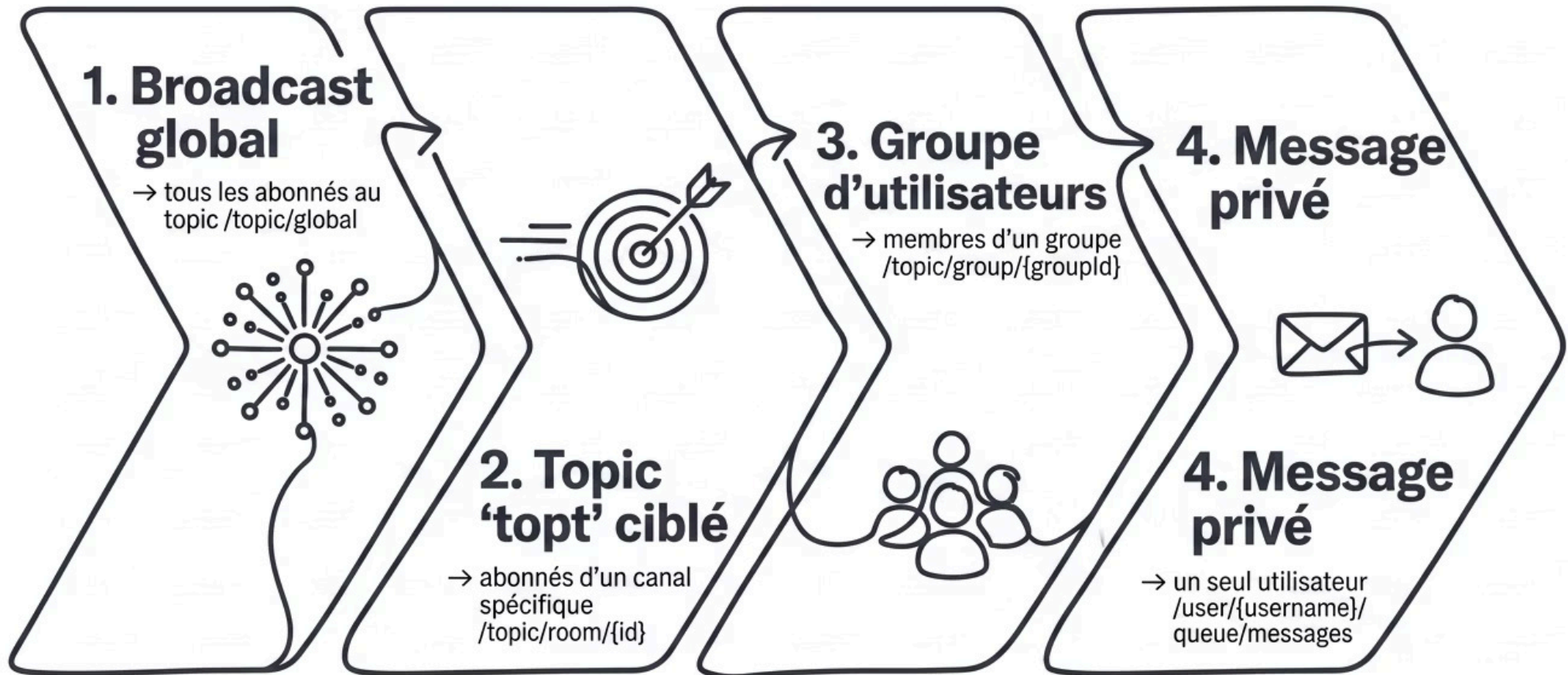
- Afficher la liste des utilisateurs connectés en temps réel
- Empêcher les connexions multiples pour un même utilisateur
- Nourrir un dashboard de supervision

Broadcasting et Messages Ciblés

Spring WebSocket offre plusieurs niveaux de granularité pour envoyer des messages — de la diffusion globale au message privé.



Les quatre niveaux de diffusion



Choisir le bon niveau de diffusion réduit la charge réseau et améliore la confidentialité des données échangées.

Broadcast global et topics ciblés

Diffusion globale

```
// Depuis n'importe quel bean Spring
@Autowired
private SimpMessagingTemplate template;

// Envoyer à TOUS les abonnés
public void broadcastAlert(AlertDto alert) {
    template.convertAndSend(
        "/topic/alerts", alert);
}

// Topic ciblé (ex: salle de chat)
public void sendToRoom(
    String roomId, MessageDto msg) {
    template.convertAndSend(
        "/topic/room/" + roomId, msg);
}
```

Message privé

```
// Message vers un utilisateur précis
public void sendPrivateMessage(
    String username, MessageDto msg) {
    template.convertAndSendToUser(
        username,
        "/queue/messages",
        msg);
}

// Dans un @MessageMapping
@MessageMapping("/chat.private")
public void handlePrivate(
    @Payload PrivateMessage msg,
    Principal principal) {
    sendPrivateMessage(
        msg.getRecipient(), msg);
}
```

Enrichir les messages avec des métadonnées

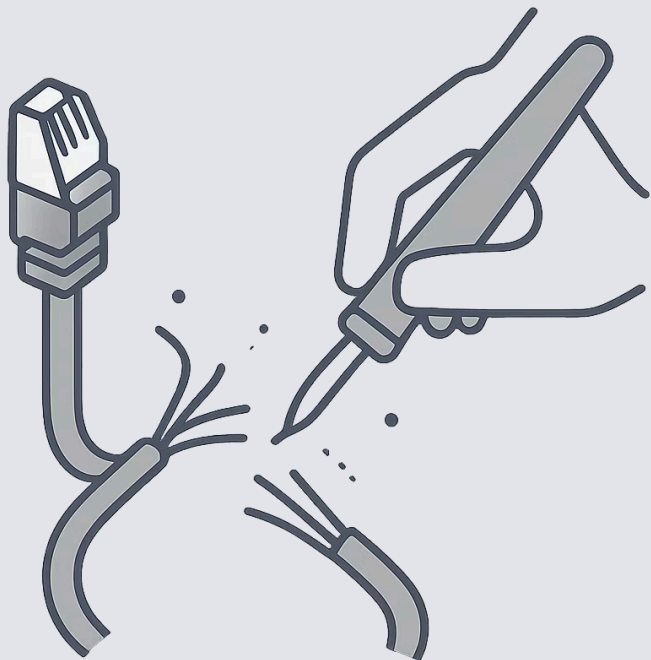
```
@Messaging("/chat.send")
public void handleMessage(
    @Payload ChatMessage message,
    Principal principal,
    @Header("simpSessionId") String sessionId) {

    // Enrichissement du message côté serveur
    message.setSender(principal.getName());
    message.setTimestamp(Instant.now());
    message.setSessionId(sessionId);
    message.setMsgId(UUID.randomUUID().toString());

    // Broadcast dans la salle
    template.convertAndSend(
        "/topic/room/" + message.getRoomId(), message);

    // Confirmation à l'expéditeur
    template.convertAndSendToUser(
        principal.getName(), "/queue/ack",
        new AckMessage(message.getMsgId(), "DELIVERED"));
}
```

 Toujours enrichir les messages côté serveur (timestamp, ID, sender) pour éviter la falsification côté client.



Gestion des Déconnexions et Reconnexions

Les connexions WebSocket sont fragiles par nature. Une stratégie robuste de reconnexion est indispensable pour une expérience utilisateur irréprochable.

Stratégies de reconnexion côté client

Reconnexion avec backoff exponentiel

```
// JavaScript — SockJS + STOMP
class ResilientWebSocketClient {
  constructor() {
    this.retryDelay = 1000; // 1s initial
    this.maxDelay = 30000; // 30s max
    this.client = null;
  }

  connect() {
    const socket = new SockJS('/ws');
    this.client = new StompClient({
      websocketFactory: () => socket,
      reconnectDelay: this.retryDelay,
      onConnect: () => {
        this.retryDelay = 1000; // reset
        this.onConnected();
      },
      onDisconnect: () => {
        this.retryDelay = Math.min(
          this.retryDelay * 2,
          this.maxDelay);
      }
    });
    this.client.activate();
  }
}
```

Pourquoi le backoff exponentiel ?

1 Évite la surcharge serveur

Si 1000 clients se reconnectent simultanément, un délai croissant étale la charge.

2 Adaptatif

Plus le serveur est indisponible longtemps, plus l'intervalle grandit (1s → 2s → 4s → ... → 30s).

3 Reset automatique

Dès la reconnexion réussie, le délai revient à sa valeur initiale.

Gérer la perte de messages lors d'une déconnexion



Stratégie de replay

Conserver un **tampon des derniers messages** côté serveur et les rejouer à la reconnexion :

Timestamp côté client

Le client envoie son `lastSeen` à la reconnexion

Buffer circulaire

Le serveur maintient un buffer des N derniers messages par topic

Replay différentiel

Seuls les messages postérieurs au `lastSeen` sont renvoyés

Implémentation du service de replay

```
@Service
public class MessageReplayService {

    // Buffer circulaire par topic (max 100 messages)
    private final Map<String, Deque<TimestampedMessage>> buffer
        = new ConcurrentHashMap<>();
    private static final int MAX_BUFFER = 100;

    public void store(String topic, Object message) {
        buffer.computeIfAbsent(topic,
            k -> new ArrayDeque<>())
            .addLast(new TimestampedMessage(
                Instant.now(), message));
        // Supprimer les anciens si dépassement
        if (buffer.get(topic).size() > MAX_BUFFER)
            buffer.get(topic).pollFirst();
    }

    public List<Object> getMissedMessages(
        String topic, Instant since) {
        return buffer.getOrDefault(topic,
            new ArrayDeque<>()).stream()
            .filter(m -> m.timestamp().isAfter(since))
            .map(TimestampedMessage::payload)
            .toList();
    }
}
```

- ❏ Pour la production, préférez **Redis Streams** ou une base de données comme PostgreSQL avec des index temporels plutôt qu'un buffer en mémoire.

Interface Utilisateur Réactive

Une UI réactive ne se contente pas d'afficher les messages — elle reflète en temps réel l'état du système : présence, typage, statut de livraison.



Indicateurs temps réel dans l'UI



Présence utilisateur

Diffuser les événements connect/disconnect via `/topic/presence`. Mettre à jour la liste des utilisateurs en ligne instantanément.



Indicateur de frappe

Envoyer un événement TYPING à chaque keystroke (debounced à 300ms) pour afficher "X est en train d'écrire..."



Statut de livraison

Accusé de réception via `/queue/ack` : un ✓ pour envoyé, ✓✓ pour reçu, ✓✓ en couleur pour lu.



État de connexion

Indicateur visuel persistant de l'état WebSocket : connecté, reconnexion en cours, hors ligne.

Gestion de l'état de connexion côté client

```
// React + @stomp/stompjs
function useWebSocketStatus() {
  const [status, setStatus] = useState('DISCONNECTED');
  const clientRef = useRef(null);

  useEffect(() => {
    const client = new Client({
      brokerURL: 'ws://localhost:8080/ws',
      onConnect: () => setStatus('CONNECTED'),
      onDisconnect: () => setStatus('DISCONNECTED'),
      onStompError: () => setStatus('ERROR'),
      onWebSocketClose: () => setStatus('RECONNECTING'),
      reconnectDelay: 5000,
    });
    client.activate();
    clientRef.current = client;
    return () => client.deactivate();
  }, []);

  return { status, client: clientRef.current };
}
```

- ✔ Exposer l'état de connexion comme un **hook React partagé** permet à tous les composants de réagir aux changements de connectivité sans duplication de logique.



Monitoring et Supervision des Connexions

En production, la visibilité sur les connexions WebSocket actives est critique pour détecter les anomalies et optimiser les performances.

Métriques clés à surveiller

∞

Sessions actives

Nombre de connexions WebSocket ouvertes à l'instant T

msg/s

Débit messages

Messages entrants et sortants par seconde

ms

Latence P99

Temps de propagation du message côté serveur (objectif < 50ms)

%

Taux d'erreur

Ratio déconnexions anormales / total connexions

Exposition des métriques avec Spring Actuator

```
// application.yml
management:
  endpoints:
    web:
      exposure:
        include: health, metrics,
          websocket, prometheus
metrics:
  tags:
    application: ${spring.application.name}
```

// Bean de métriques custom

`@Component`

```
public class WebSocketMetrics {
```

```
    private final MeterRegistry registry;
```

```
    private final AtomicLong activeSessions
        = new AtomicLong(0);
```

```
    public WebSocketMetrics(MeterRegistry registry) {
        this.registry = registry;
        Gauge.builder("websocket.sessions.active",
            activeSessions, AtomicLong::get)
            .description("Sessions WebSocket actives")
            .register(registry);
    }
```

```
    public void incrementSessions() {
        activeSessions.incrementAndGet();
        registry.counter("websocket.sessions.total")
            .increment();
    }
}
```

Stack de monitoring recommandée

Micrometer

Abstraction métriques
intégrée à Spring Boot

Prometheus

Collecte et stockage des
métriques time-series

Grafana

Dashboard visuel avec alertes configurables

Dashboard de supervision WebSocket

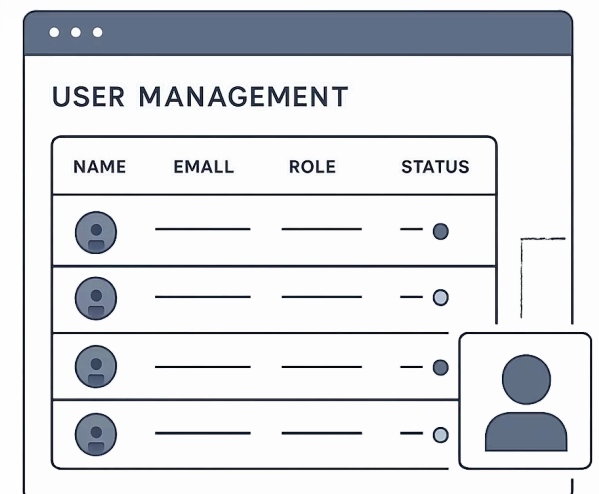
Endpoint de supervision dédié

```
@RestController
@RequestMapping("/api/ws-admin")
public class WebSocketAdminController {

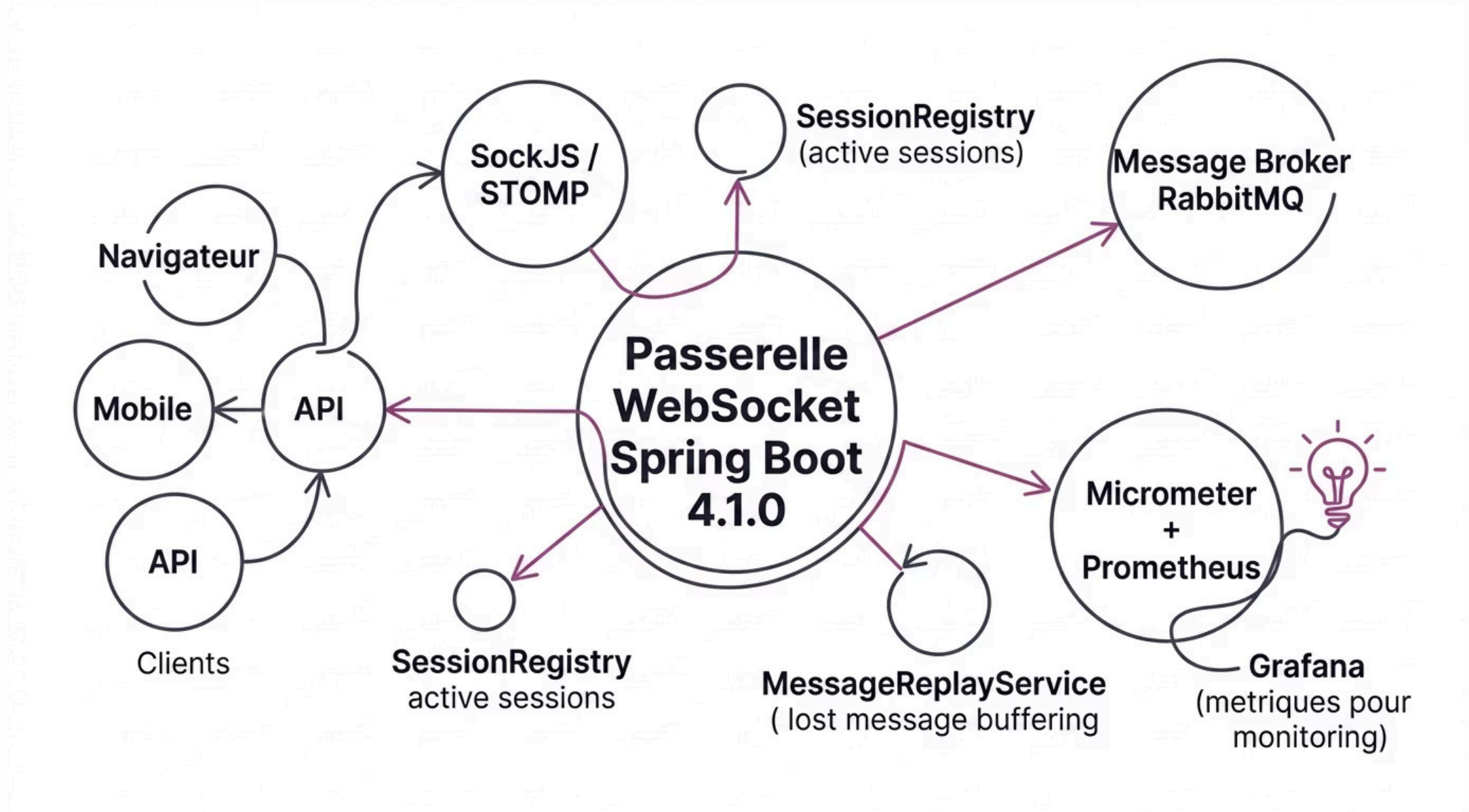
    @Autowired
    private SessionRegistry sessionRegistry;

    @GetMapping("/sessions")
    public WebSocketStats getStats() {
        return WebSocketStats.builder()
            .activeSessions(
                sessionRegistry.countActiveSessions()
            )
            .connectedUsers(
                sessionRegistry.getActiveUsers()
            )
            .uptimeSeconds(
                sessionRegistry.getUptimeSeconds()
            )
            .build();
    }

    @DeleteMapping("/sessions/{sessionId}")
    public void forceDisconnect(
        @PathVariable String sessionId) {
        sessionRegistry.forceDisconnect(sessionId);
    }
}
```



Récapitulatif de l'architecture complète



Cette architecture garantit **scalabilité horizontale**, **résilience aux déconnexions** et **observabilité complète** en production.

Bonnes Pratiques & Points Clés



À retenir pour la production



Sécurité des sessions

Authentifiez toujours les connexions WebSocket via Spring Security. Associez chaque session à un `Principal` et vérifiez les autorisations sur chaque `@MessageMapping`.



Broker externe obligatoire

Dès que vous déployez plusieurs instances, le broker in-memory ne suffit plus. Utilisez RabbitMQ ou Redis Pub/Sub comme broker STOMP partagé.



Backoff exponentiel

Implémentez toujours une stratégie de reconnexion avec délai croissant côté client pour protéger votre infrastructure lors des redémarrages.



Monitoring dès le début

Intégrez Micrometer et exposez les métriques WebSocket custom dès le développement. Les problèmes de performance se détectent tôt ou trop tard.

Prochaines étapes

Pour aller plus loin

1 Tests de charge WebSocket

Tester avec Gatling ou k6 — simuler des milliers de connexions simultanées et valider la tenue en charge.

2 WebSocket + Spring Security OAuth2

Authentification JWT sur la handshake WebSocket et autorisation fine par topic et par utilisateur.

3 Reactive WebSocket avec Spring WebFlux

Alternative non-bloquante basée sur Project Reactor pour des performances maximales avec un minimum de threads.

